基于 OpenMP 的 Sobel 图像边缘检测

并行程序设计与性能分析

学号：待填写　姓名：待填写

（学院、专业及班级：待填写）

实验日期：2026 年 10 月 25 日

摘要　为研究共享内存并行化对图像边缘检测的实际收益，采用 C++ 自行实现串行和 OpenMP Sobel 算法。实验固定使用一张 4096×4096 灰度合成图像，统一卷积核、零边界处理和幅值截断规则，按行静态划分任务。通过独立参考实现、手算样例及逐像素比较验证正确性。在 4 核等效 CPU 配额的云环境中，分别测量串行及 1、2、4 线程配置，每组预热 3 次、正式运行 30 次，仅统计核心计算时间。串行和 4 线程平均耗时分别为 65.850 ms 和 34.074 ms，获得 1.933 倍加速。2 线程到 4 线程的收益明显减小，4 线程效率为 48.31%。结果表明该实现能够受益于并行化，但增加线程数量并不产生等比例收益；云环境中的计时波动也限制了结论的外推范围。

关键词　Sobel 算子；图像边缘检测；OpenMP；数据并行；加速比；并行效率

1 引言

边缘是图像中灰度发生明显变化的区域，可为轮廓识别和后续视觉分析提供信息。Sobel 算子通过两个 3×3 卷积核估计水平和垂直方向的一阶梯度，再合成梯度幅值[1]。像素数增大时，重复的局部计算构成了适合并行处理的工作负载。

对固定输入，每个输出像素仅依赖输入邻域，不依赖其他输出像素。因此可以在共享内存中按行划分任务。本文只回答串行与并行的时间差异，以及线程数对时间、加速比和效率的影响；不改变图像尺寸，不比较其他算子或调度策略。

2 程序总体设计

程序分为图像 I/O、串行核心、并行核心、正确性验证和性能统计五个模块。采用标准二进制 P5 PGM 格式保存 8 位灰度像素，直接在 C++中读写，不依赖 OpenCV。PNG 用于展示，绘图与报告脚本不参与核心计时。

整体流程为：读取固定输入→分配输入和输出缓冲区→验证串行及各线程输出→预热→交错执行四种配置并计时→保存原始 CSV→计算统计量并生成图表。图像读写、内存分配、结果比较及绘图均位于计时区间之外。

serial.cpp 和 parallel.cpp 分别提供串行与并行入口；sobel.hpp 保存共同的单行计算逻辑；io.cpp 负责 PGM 读写；verify.cpp 实现独立参考卷积；benchmark.cpp 负责正式计时；run_experiment.py 固定输入、收集环境与汇总数据。

3 串行与并行算法设计

3.1 Sobel 核心计算

采用如下两个卷积核，图像坐标中 y 向下递增，垂直核的符号不影响幅值[1]:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

对内部像素累加得到 Gx 与 Gy，随后执行以下输出规则:

$$G = \sqrt{(G_x^2 + G_y^2)}$$

$$\text{output} = \text{floor}(\min(255, G))$$

最外层一圈输出直接置 0，不计算越界邻域；这与对输入补零后再计算边界不同。梯度以 int 存储，单方向绝对值不超过 1020，平方和不会溢出 32 位有符号整数。开平方使用 double，饱和后转换成 uint8_t，不做归一化或阈值二值化。

3.2 串行实现

外层按 y 从 0 到 H−1 逐行遍历，内层遍历有效列，对每行调用共同的计算函数并写入对应输出行。每次调用覆盖整个输出，包括边界。时间复杂度为 O(WH)，输入和输出缓冲区的空间复杂度均为 O(WH)。编译器仍可进行常规优化，串行程序不创建 OpenMP 工作线程。

3.3 OpenMP 实现

在外层行循环上使用 parallel for 、schedule(static)和 num_threads(p)。输入、输出对象为 shared；高度为 firstprivate；循环索引以及单行函数内的梯度、指针和列索引均为线程局部变量。default(none)要求显式声明共享关系。

各行的像素数量和计算量基本一致，静态调度可将连续行块分给线程，避免动态分派成本。输入仅被读取，各线程写入不同输出行，不需要锁或原子操作；parallel for 末尾的隐式屏障确保计时结束时所有线程均完成。串行和并行复用单行公式，避免算法差异干扰性能对比。

4 实验环境与实验方法

表 1 实验环境与固定配置

Table 1 Experimental environment and fixed configuration

项目	实测配置
处理器	Intel Xeon Platinum 8573C（KVM 虚拟环境）
可见 CPU / CPU 配额	5 个逻辑 CPU；cpu.max = 400000 100000，等效 4 核
操作系统 / 架构	Debian GNU/Linux 13 / x86_64
内存限制	32 GiB（cgroup memory.max）
编译器 / OpenMP	G++ 14.2.0；libgomp；-fopenmp，_OPENMP=201511
统一编译选项	-O3 -std=c++17 -fopenmp -Wall -Wextra -Wpedantic
输入图像	4096×4096，8 位灰度，自行生成且全实验固定
线程配置	1、2、4；实验一固定 4 线程
运行策略	关闭动态线程；close 绑定；cores 位置；PASSIVE 等待
计时与重复	omp_get_wtime；每组预热 3 次、测量 30 次

4.1 固定输入与可复现性

因未提供自然图像，本实验使用脚本生成固定灰度合成场景：线性灰度背景、亮矩形、暗圆、三角形和条纹。它能展示多个方向的轮廓与缓慢灰度变化，不代表自然图像数据集。原始 PGM 和预览 PNG 均随提交保留，所有计时组读取同一份 PGM。

输入 PGM 的 SHA-256：7630efcd74453761e2c346189ee5ca6c2e348c7d4854f0c89ab10aabee4e0ced

4.2 两项性能实验与计时口径

实验一仅比较串行与预先选定的 4 线程配置。实验二比较 OpenMP 的 1、2、4 线程配置，统一以串行均值为基准。容器有效 CPU 配额为 4 核，因此不把 8 线程作为主测试对象。四组数据来自同一次基准运行，实验一复用其中的串行和 4 线程数据，不另行改变实验条件。

在正式测量前，为各配置预热 3 次并核验实际线程团队数量。正式阶段共 30 轮，每轮以固定随机种子 20261025 打乱四种配置的执行顺序，以减轻顺序与热状态偏差。每个样本只计一次完整核心调用，包括边界写入、OpenMP 线程唤醒、循环分配和末尾同步；不包含文件读写、内存分配或正确性比较。预热后测量反映稳态执行，不能据此估计冷启动开销[2]。

执行时间取 30 个样本的算术平均值，离散程度取样本标准差（分母 n−1）。加速比 Sp=Tserial/Tparallel(p)，效率 Ep=Sp/p。图中误差线表示标准差，不是置信区间。原始计时值、运行顺序、环境信息及输入校验值均保存，未删除慢样本。

4.3 正确性验收

verify.cpp 使用独立的双层 3×3 系数循环作为参考，覆盖 7 种尺寸（含 1×1、窄图和非方图）、5 类输入（全零、常数、水平斜坡、阶跃、固定种子随机值）、串行及 3 种线程配置，共 140 项逐像素比较。另用 5×5 水平斜坡手算验证中心 Gx=24 、 Gy=0 、 输出 =24 。普通构建与 AddressSanitizer/UndefinedBehaviorSanitizer 构建均通过。

对完整 4096×4096 输入，各配置预热结果与串行输出相同，120 次正式计时调用的输出逐字节比较均为 0 差异像素。所有图像尺寸一致，边界均为 0。独立参考和手算用于避免“两个版本共享同一错误但仍一致”的问题；可视图只提供直观补充。

5 实验结果与分析

5.1 实验一：串行与 4 线程对比

表 2 串行与 4 线程的核心计算性能

Table 2 Core computation performance: serial versus four threads

实现	次数	均值/ms	标准差/ms	加速比
串行	30	65.850	10.342	1.000
OpenMP 4 线程	30	34.074	6.033	1.933

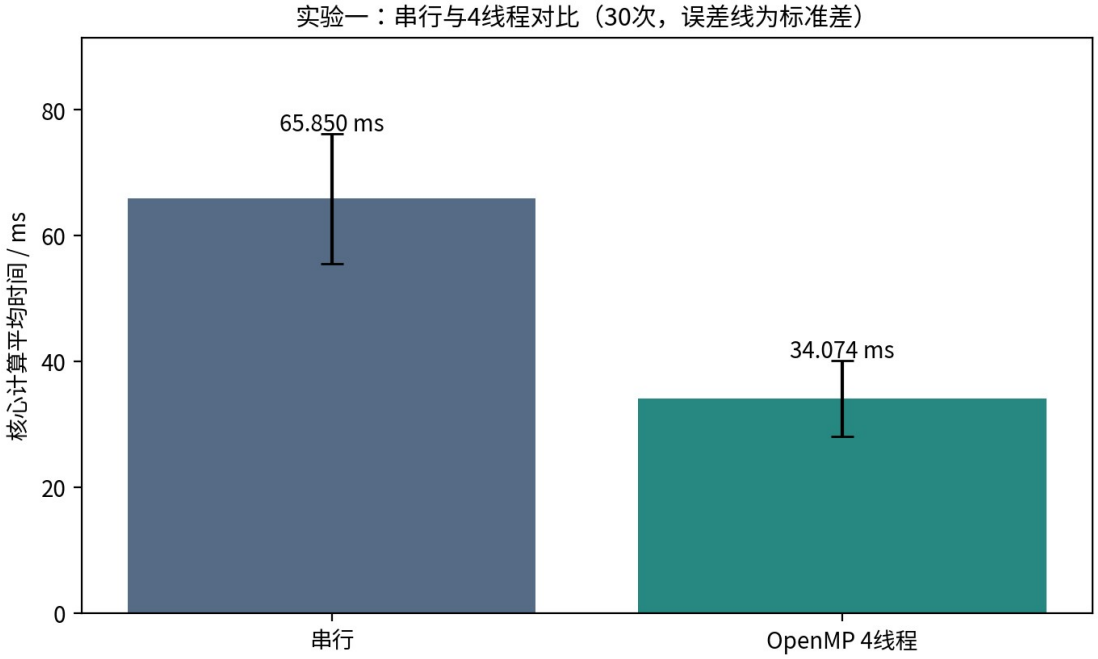


图 1 串行与 4 线程耗时对比（误差线为样本标准差）

Fig. 1 Serial and four-thread runtime (sample standard deviation)

如表 2 及图 1 所示，4 线程将核心平均时间由 65.850 ms 降至 34.074 ms，减少 48.25%，加速比为 1.933。因此在当前输入和运行条件下，并行实现确实提高了核心计算效率。该结果不包含 I/O，不能直接当作端到端程序的加速比。

串行样本范围为 57.450～111.574 ms，4 线程为 25.615～55.297 ms，说明存在运行波动。1 线程 OpenMP 均值为 64.771 ms，略低于串行，差异约 1.64%，小于组内波动。不能将这种微小差异解释成单线程并行化的稳定收益，也不能由此断言线程管理没有开销。

OpenMP 需要执行线程团队管理、任务分配和屏障同步，这些开销已计入并行核心时间。固定大图能够提供足够的计算工作来摊薄开销；本实验未单独拆分这些成本，也未改变图像尺寸，因此不对小图或冷启动的性能作额外结论。

5.2 实验二：线程数量的影响

表 3 不同线程数的性能（统一以串行均值为基准）

Table 3 Performance by thread count, relative to the serial mean

线程数	均值/ms	标准差/ms	加速比	效率/%
1	64.771	7.304	1.017	101.67
2	37.097	7.501	1.775	88.75
4	34.074	6.033	1.933	48.31

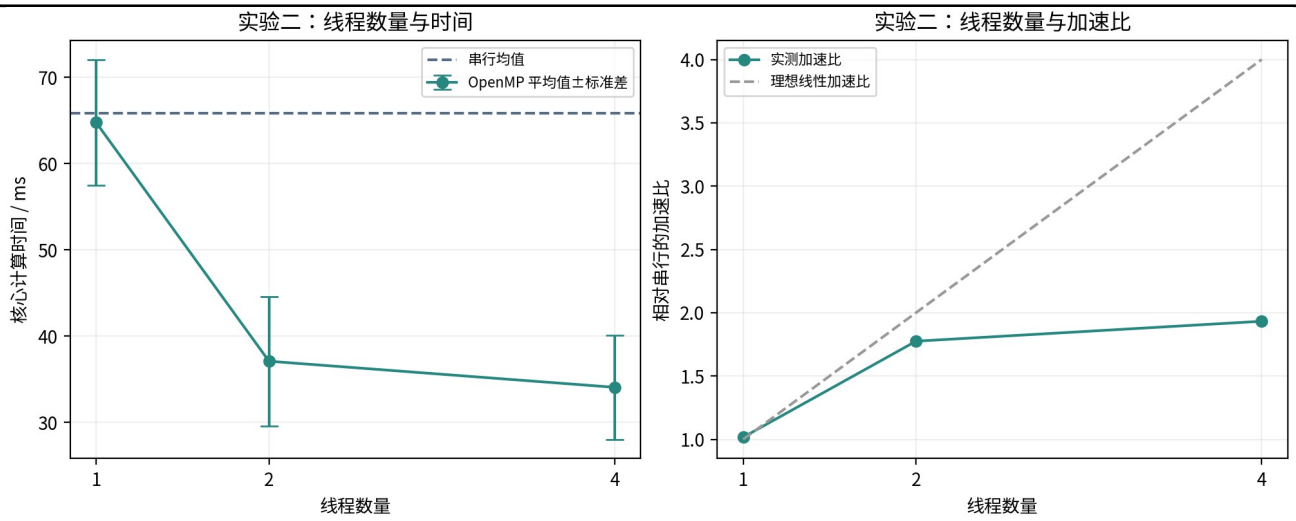


图 2 线程数量与时间、加速比的关系

Fig. 2 Thread count versus runtime and speedup

由表 3 及图 2，OpenMP 从 1 线程增至 2 线程，平均耗时由 64.771 ms 降至 37.097 ms；增至 4 线程后进一步降至 34.074 ms。线程增加时平均时间下降，但 2 到 4 线程只进一步减少 8.15%，未达到线程翻倍、时间减半的理想情况。

相对于串行，加速比从 1.017 增至 1.775 和 1.933，在所测线程范围内持续增加，但增长明显放缓。2 线程效率为 88.75%，4 线程效率降至 48.31%。1 线程的效率约 101.67%，略超 100%源于分开测得的均值及波动，并不表示超出物理限制的并行收益。

静态分配使各线程行数基本均衡，但线程团队切换和同步仍有固定成本，输入邻域读取和输出写入也会竞争缓存与内存资源。云主机的资源共享、虚拟化调度和 CPU 频率变化可能增加波动。这些均是可解释收益递减的候选因素；本实验没有硬件计数器或分项开销测量，不能确定各因素的贡献。

基准计时前后 cgroup 的 nr_throttled 均为 13，throttled_usec 均为 380293，本次计时窗口未观察到新增配额节流。因此，虽然配额决定主实验最多使用 4 线程，也不能将 4 线程扩展受限直接归因于本次触发配额限流。虚拟环境的其他竞争来源仍未被排除。

结论只覆盖 1、2、4 线程。在这一范围内已出现收益递减，但未测量更多线程，不能把“继续增加必然变慢”当作实测事实。4 线程在当前均值上最快；2 线程以较少的线程获得较高效率，体现了资源利用率和最短时间之间的取舍。



图 3 固定输入及串行、4 线程边缘结果

Fig. 3 Fixed input and serial/four-thread edge outputs

6 总结

本文完成了 C++串行与 OpenMP Sobel 实现，采用按行静态调度、独立输出行和统一幅值规则。独立参考测试、手算检查、内存检查以及完整图像逐像素比较均通过。固定输入、相同编译选项和核心计时口径保证了两项实验的基本可比性。

当前环境下，4 线程相对串行取得 1.933 倍平均加速；从 2 线程增加到 4 线程仍有收益，但效率从 88.75%降至 48.31%。因此，数据并行可以缩短 Sobel 核心时间，线程数增加却不会带来等比例提升。

局限在于只使用一张合成灰度图、一个云端运行环境和一套静态调度策略。计时具有波动，未进行硬件事件归因；不测量 I/O 端到端时间，也没有测量 4 线程以上的性能。结果不能直接推广到自然图像集合、其他处理器或其他算法。

7 参考文献

[1] Huamán A. Sobel derivatives[EB/OL]. OpenCV 4.10.0 documentation, 2024[2026-10-25]. https://github.com/opencv/opencv/blob/4.10.0/doc/tutorials/imgproc/imgtrans/sobel_derivatives/sobel_derivatives.markdown.

[2] Free Software Foundation. GNU Offloading and Multi Processing Runtime Library: Runtime Library Routines; Environment Variables[EB/OL]. GCC 14.2.0, 2024[2026-10-25]. https://github.com/gcc-mirror/gcc/blob/releases/gcc-14.2.0/libgomp/libgomp.texi.

附：复现与文件说明

代码目录含 Makefile 与 run_experiment.py；在代码目录执行 make test 验证核心，再执行 python3 run_experiment.py 可重新编译、测试、生成固定输入并完成两项实验。脚本将覆盖实验结果文件，因此重新测量后应同步更新报告。报告生成脚本保存在仓库根目录 temp/build_report.py。

实验结果目录保存原始 120 条计时记录、汇总 CSV、环境元数据、输入图像、输出图像与图表。根目录 temp 仅存放原始模板、构建产物、文档生成工具及中间文件，不属于课程提交正文。

学号、姓名、学院与班级尚未提供，保留“待填写”。代码与报告包含 AI 辅助生成内容，需由提交者结合课程要求审阅、理解并据实署名。原模板中的无关案例与“全部由本人独立完成”声明均已替换。